

BUSINESS ANALYST INTERN TECHNICAL ASSESSMENT

Question Set: C

Congratulations on clearing the aptitude round. This assessment has four sections, all built on a single CSV. Section 2 is a substantial CASE STUDY read its briefing carefully. The mini-project (Section 4) is the only open-ended deliverable; everything else has specific, verifiable answers.

Suggested time: 4 hours, submit a single ZIP. Read all footnotes before starting them document real data quirks that will sink surface-level analysis.

The Dataset

A single CSV (hotel bookings flat.csv) of 12,000 booking records from a hotel-booking platform. Each row is one booking; customer and property attributes are denormalized into the row. 28 columns:

- **Booking id, customer id, property id** — Primary key + foreign keys
- **Customer name, customer segment, customer signup date, customer home city, customer loyalty tier** — Customer attributes (denormalized into every row)
- **Property name, property city, property star rating, property type, property total rooms** — Property attributes (denormalized into every row)
- **Booking date, check-in date, checkout date** — Key dates
- **Room type, num rooms, nights** — Room details
- **Booking channel** — Direct Website / OTA / Travel Agent / Corporate Portal
- **adr, discount amount, coupon code, total amount, payment method** — Pricing & payment (total amount = $\text{adr} * \text{nights} * \text{num rooms} - \text{discount amount}$)
- **Booking status** — Completed / Cancelled / No-Show
- **Review rating, review date** — Review info (null if no review left)

Important Footnotes read before starting

Footnote 1 Invalid stays: Some rows have checkout date on or before check-in date. Treat as data errors.

Footnote 2 Booking before account: Some rows have booking date earlier than customer signup date a temporal-integrity issue.

Footnote 3 Zero rooms: Some rows have num rooms = 0 (carry total amount = 0). These are erroneous.

Footnote 4 Property names repeat across cities: A handful of property name values appear in multiple cities with DIFFERENT property ids. NEVER group on property name always use property id.

Footnote 5 Reviews on Cancelled bookings: A small number of Cancelled bookings carry a review rating, which is impossible. Flag these.

Footnote 6 Review scale mismatch: review rating is on 1-5 for Individual and Group customers, but on 1-10 for corporate customers. Without normalization, any cross-segment rating average is wrong.

Footnote 7 customer loyalty tier 'None' parsing trap: The tier column has 4 valid values including the literal string 'None' (meaning 'no tier'). Default CSV readers (pandas read csv, etc.) will silently convert 'None' to a missing value, making ~46% of your tier data disappear. Use keep default Na=False (pandas) or the equivalent in your tool.

Footnote 8 Realized vs. booked revenue: Only booking status = 'Completed' counts as realized revenue. Cancelled and No-Show rows still carry a total amount (the would-have-been-charged amount); summing all rows overstates revenue by 30%.

Section 1 Data Quality Checks (quick)

- C1.** How many bookings have num rooms = 0? What does this imply about the upstream pipeline? (1 line)
- C2.** How many bookings have booking date earlier than customer signup date? Give two plausible explanations — not all of which mean 'data corruption'.
- C3.** What % of bookings have no review (review rating missing)? Break down by booking status — does the pattern make sense?

Section 2 Case Study: The VIP-vs-Volume Strategy

Briefing: The Head of Growth has ₹50 lakhs of incremental budget for next quarter and must split it between two strategies: (1) acquiring NEW customers via paid channels (assume ₹3,000 average CAC), or (2) retaining at-risk EXISTING customers via targeted discount offers (assume ₹500 in discount value per retained customer per year). Marketing is pushing for 'double down on VIP retention.' Operations is skeptical that 'VIP' is even well-defined here. Your job: define VIP rigorously, characterize their behaviour from the data, do the acquisition-vs-retention math under the stated assumptions, and recommend a budget split with a sensitivity analysis.

What we expect: You must include at least TWO labelled visualizations: one overlap diagram (Venn or UpSet plot) for the three VIP definitions, and one comparison chart of VIP vs non-VIP behaviour. The recommendation MUST be derived from the math — not from a general view about 'retention is better.' Show the break-even computation.

C1 Define VIP Three Ways (visualization required).

Identify the top 10% of customers under EACH of three definitions, computed only over Completed bookings: (i) by total spend (sum of total amount), (ii) by booking count, (iii) by lifetime value rate (total spend ÷ months since customer signup date). Each definition gives you a set of ~80 customers. Compute the OVERLAP: how many customers appear in all three top-10% sets? In exactly two? In only one?

C2 VIP Risk Profile.

Pick ONE definition you will use going forward (justify briefly). For that VIP cohort, compute: (a) share of platform realized revenue, (b) cancellation rate vs non-VIP, (c) discount intensity vs non-VIP. Produce a comparison chart. State whether VIPs are LOWER-risk, HIGHER-risk, or comparable to non-VIPs, and whether the platform is currently giving them MORE or FEWER discounts than non-VIPs.

C3 Acquisition Math.

Compute the acquisition arithmetic: ₹50L at ₹3,000 CAC =? new customers. Assuming a new customer reaches the platform's AVERAGE customer lifetime spend over time, what is the gross lifetime revenue from these acquisitions? Year-one revenue assumption: an acquired customer generates ~20% of their lifetime spend in the first year. Compute Year-1 revenue from the acquisition spend.

C4 Retention Math.

Compute the retention arithmetic: ₹50L at ₹500 / retained customer =? customers retained. How many customers can the budget theoretically retain vs. the actually retained given in the base. State the assumption that drives this number.

Section 3 SQL Challenge

Tests SQL fluency AND schema understanding. The dataset is denormalized; your first task is to design a normalized schema. The queries are written against the schema YOU design.

Schema Design Task (worth as much as the queries):

Design a normalized schema by writing CREATE TABLE statements for at least three tables (customers, properties, bookings, optionally reviews). For every column specify:

- SQL data type (INTEGER, VARCHAR(n), DECIMAL(p,s), DATE, etc.) think precision for money and length for free text;
- primary key, foreign key, or neither plus NOT NULL where appropriate;
- ONE non-trivial CHECK or UNIQUE constraint addressing one of the footnotes (e.g., CHECK(checkout date > check-in date); UNIQUE(property name, property city));
- ONE column to index for speeding up your queries below, with a 1-2 line justification.

SQL Queries (against your normalized schema, or against the flat CSV loaded as one table state which):

C-Q1.

Use ROW NUMBER () OVER (PARTITION BY customer id ORDER BY check-in date) to number each customer's Completed bookings. Compute average total amount for booking sequence 1 (first), 2 (second), and 3 or later. Output: sequence label, avg total, n.

C-Q2.

Top 5 properties by AVERAGE review rating, with at least 20 reviews. CRITICAL: Footnote 7 normalize Corporate reviews from 1-10 to 1-5 ($\div 2$) BEFORE averaging. Output: property name, property city, num reviews, normalized avg rating.

Section 4 Mini-Project (build something + integrate a free API)

The only open-ended part. Build a small WORKING tool or notebook that enriches the dataset using a live, free, no-key public API and produces one concrete, quantified insight. You MAY (and should) use AI assistants we expect it. But the deliverable must be a working artifact YOU wired together, debugged, and can explain. We are assessing how well you USE AI to BUILD, not whether AI can answer a prompt.

Set C Multi-Currency Revenue Reporter

API: Frankfurter FX API free, no key. Endpoints: <https://api.frankfurter.app/latest?base=INR&symbols=USD,EUR,GBP> and <https://api.frankfurter.app/2024-06-01?base=INR&symbols=USD> for historical dates.

What to build: Build a tool that converts realized revenue into a chosen currency. Bonus: for monthly trends, use each MONTH'S historical FX rate (not today's spot) so an international stakeholder sees the FX-adjusted picture. Quantify how much of the apparent monthly trend disappears (or reverses) once you remove the rupee's movement.

Every mini-project must include ALL of the following:

1. A real call to the assigned free API, with error handling timeouts or missing data should NOT crash the run. Show this in the code.
2. A correct merge of the API data back onto the bookings (watch join keys, dates, city country/coords mappings).
3. ONE non-obvious, quantified insight the dataset ALONE could not produce a number or chart, stated in 3-4 sentences.
4. A 6-line README: what it does, how to run it, one design decision, one limitation.
5. A specific note (≤ 8 lines) on where you used AI and what you had to FIX in the AI's output vague notes lose marks; specific notes win.

Deliverables & How We Grade

Submit a single ZIP containing:

1. answers.pdf (or .docx): your CREATE TABLE statements + index justification, your SQL queries pasted as text, and the case-study answers with embedded visualizations.
2. code/ folder: your analysis notebook/script + SQL files.
3. project/ folder: working mini-project code + README + the insight + AI-usage note.